

Por qué tu IDE y tu Compilador necesitan IAs opuestas: Arquitectura de Enrutamiento Híbrido

Estantería: Art de Côté

Subtema: Inteligencia Artificial

mercedev.es — 2026-06-15 | Epic 9

Por qué tu IDE y tu Compilador necesitan IAs opuestas

Cuando integras Inteligencia Artificial Generativa en todo el ciclo de vida de tu software, cometes rápidamente el primer error de novato: usar el mismo proveedor (como OpenAI o Google Gemini) para todo.

En **mercedev.es**, el ecosistema está gobernado por IA en dos frentes masivos: 1.

El Entorno de Desarrollo (IDE Antigravity): Donde programo, refactorizo y debato sobre arquitectura. 2. **El Pipeline de Compilación (merci-publish):** Donde un ejército de agentes genera metadatos, saludos y resúmenes para decenas de artículos en tiempo de compilación.

Si usas la API de Google para ambos, te quedarás sin cuota gratuita (Error HTTP 429 - Too Many Requests) antes de terminar el día. ¿La solución? Diseñar dos **Pilas Híbridas de Enrutamiento (Hybrid Stacks)** con reglas de negocio estrictamente invertidas.

1. El Router del IDE: Inteligencia Máxima → Supervivencia Local

Cuando abro la terminal y escribo mi alias de desarrollo, se levanta un servidor de `litellm` configurado con una directiva muy clara: **Priorizar el coeficiente intelectual**.

En programación, un error de contexto de la IA te cuesta horas de depuración. Por tanto, el enrutador del IDE funciona así: - **Intento Primario (Cloud):** Todas las consultas van directas a *Google Gemini 2.5 Flash* (o modelos equivalentes de altísima capacidad). - **Fallback de Emergencia (Local):** Si Google me bloquea por límite de tokens o se cae el internet, `litellm` intercepta el error HTTP y deriva la consulta silenciosamente a **LM Studio (qwen2.5-coder)** ejecutándose en mi propio hardware (puerto 1234).

[!TIP] **El resultado:** Pierdo un porcentaje de inteligencia o contexto, pero sigo programando sin interrupciones y a coste cero. Es pura "Degradación Elegante" (Graceful Degradation).

2. El Router del Compilador: Ahorro Máximo → Rescate Cloud → Circuit Breaker

El pipeline de compilación estática es una bestia muy distinta. Generar 100 meta-descripciones SEO o saludar a los lectores de un artículo no requiere un modelo de 175 mil millones de parámetros. Gastar tokens de la nube en esto es un crimen arquitectónico.

Por eso, el script `merci-brain.py` (el Lóbulo Frontal del compilador) invierte las reglas: - **Intento Primario (Local):** Todas las peticiones bombardean implacablemente a **Ollama (qwen2.5-coder)** en el puerto 11434. Es rápido, privado y 100% gratuito. Cero desgaste de cuotas. - **Red de Seguridad (Cloud):** Si el servidor de Ollama se cae (o si el *Chaos Monkey* lo sabotea durante las pruebas de CI/CD), el sistema redirige el tráfico hacia el proxy de Gemini en la nube para asegurar que la web se pueda desplegar sí o sí. - **El Cortacircuitos (Circuit Breaker):** ¿Y si Gemini también falla? A diferencia del IDE, el compilador *no sigue buscando alternativas*. Activa un Cortacircuitos: suspende las llamadas a la red e inyecta una **Contingencia Estática** (ej. `[Fallback] Bienvenido al artículo...`) en el resto del sitio web.

[!IMPORTANT] **El resultado:** El pipeline de integración continua finaliza con *Exit Code 0*. La web se publica a tiempo con latencia 0ms, sacrificando personalización dinámica a cambio de resiliencia absoluta.

Conclusión

Una estrategia DevSecOps madura no trata a la IA como un bloque monolítico. Obliga a que cada pieza del engranaje use los recursos de forma asimétrica.

- Tu IDE debe **Gastar Nube para ganar Tiempo**.
- Tu Compilador debe **Gastar Hardware para ganar Resiliencia**.

Y al final, un buen Cortacircuitos siempre será tu mejor ingeniero de guardia.